

BhatBot — Research Ideas & Hardware Roadmap

Companion to the JARVIS-parity software build (see `.claude/plans/quirky-strolling-planet.md`). This document captures (1) research directions to explore with **Manus** and **Claude**, and (2) the **hardware** ideas kept out of the software plan's scope but retained here for later. Software workstreams W1–W7 are implemented; this is the forward-looking layer.

Part 1 — Research directions (Manus + Claude)

These are open-ended investigations, not committed engineering. Each names the tool best suited to drive it: **Manus** for autonomous, long-horizon agentic exploration; **Claude** for reasoning-dense design, analysis, and code.

1. Closed-loop self-improvement from the trace corpus

Now that every turn is logged with full cost/energy telemetry (W2) and mined into SFT/preference pairs (W5), the open question is *what actually moves the needle*: LoRA on weights, prompt optimization (DSPy/GEPA), agentic-logic edits, or router tuning.

- **Claude**: design the ablation — hold the eval suite fixed, vary one surface at a time, attribute win-rate deltas. Define the preference-pair quality bar (when is a correction a real signal vs noise?).
- **Manus**: run the sweep autonomously overnight — export, train, A/B, log, repeat across hyperparameters — and surface the Pareto front of quality vs latency vs \$.
- **Open question**: at personal data scale (hundreds, not millions of turns), does LoRA beat a well-tuned prompt? Find the crossover point.

2. Knowledge-graph reasoning depth

W4 added entity/edge memory with 2-hop traversal. Research: how deep is useful before noise dominates?

- **Claude**: design multi-hop eval questions over the real graph; compare flat-embedding recall vs graph traversal vs hybrid. Study edge-type taxonomy — is a free-text predicate enough, or does a closed relation schema (works_on, uses, depends_on, located_in) measurably help retrieval?
- **Manus**: stress-test extraction quality at scale — feed a month of memories, measure triple precision/recall against a hand-labeled set.
- **Open question**: can the graph drive *proactive* behavior (notice "project P uses tool T, T just shipped a breaking change") rather than only answer queries?

3. Context-rot economics

W1 cut per-turn tool injection from ~50 to ~12. Research the full curve.

- **Claude**: model the trade — tokens saved vs capability loss from a mis-selected tool. Derive the optimal `k` and `minScore` from the audit log empirically rather than the current hand-set 12/0.18.
- **Manus**: replay historical turns with varying `k`, measure task-success regressions, find the knee.
- **Open question**: does a learned reranker (tiny local model) beat raw cosine for tool selection?

4. Multi-agent division of labor (A2A)

W7 standardized handoffs. Research what topology actually helps.

- **Claude:** design experiments comparing single-agent vs orchestrator+specialists on real multi-step tasks; measure where handoff overhead pays off vs where it just adds latency.
- **Manus:** prototype an *external* A2A endpoint (the drop-in remote branch) — e.g. a research agent on a Princeton compute node — and benchmark a real cross-machine handoff.
- **Open question:** what's the right granularity — three persistent specialists, or dynamic per-task agents?

5. Voice/persona faithfulness under compression

The JARVIS persona + brevity rules fight the ElevenLabs char budget.

- **Claude:** study how much persona survives aggressive brevity; find the minimal prompt that holds character at one sentence.
- **Manus:** A/B persona variants on recorded sample scripts, score "feels like JARVIS" via blind rating.

6. Proactive autonomy with a safety budget

W3's risk tiers make selective autonomy safe. Research: how much *unprompted* action is welcome?

- **Claude:** design a "proactivity budget" — the agent may take N auto-tier actions/day unprompted (pre-brief prep, inbox triage) but logs each for review; study the annoyance/usefulness curve.
- **Open question:** can the correction signal (W5) auto-tune the proactivity threshold per user mood/context?

Part 2 — Hardware roadmap (retained, not in current software scope)

Ranked by leverage. These are deliberately deferred — the software build comes first — but kept here so the path is explicit.

A. Dedicated Mac Mini M4 — always-on local node · HIGHEST LEVERAGE

The single biggest operational gap is the Mac sleeping on battery: queued commands run stale, heavy local tools (Ollama, OmniParser, simulate, MLX fine-tuning from W5) are unavailable, and `caffeinate` hacks are brittle.

- **What:** a ~\$600 Mac Mini M4, always plugged in, never sleeps, dedicated 16-core NPU for local inference + the W5 MLX LoRA runs.
- **Why now-ish:** it turns "local-first" from aspiration into reality — the router's local rungs become reliably available, and fine-tuning has a home that isn't the daily-driver laptop.
- **Signal:** Stanford's OpenJarvis offers a Mac Mini as its efficiency-leaderboard prize — the community's reference local-AI platform.

B. ESP32-S3 wake-word satellites · CHEAP, HIGH AMBIENT PAYOFF

Small mic boards (ESP32-S3) running a wake-word model (Porcupine/Vosk), connecting to BhatBot's cloud endpoint over WiFi.

- **What:** a few units around the apartment/dorm → walk into any room and BhatBot is listening, no phone/Mac needed.

- **Why:** delivers the "JARVIS is always around" presence that no software change can — closes the gap between "fast chatbot" and "ambient assistant."
- **Reference:** Open Interpreter's 01 project is the leading open voice interface across desktop/mobile/ESP32.

C. Home Assistant integration · LIFESTYLE UPGRADE

A Raspberry Pi Home Assistant instance controlling lights/thermostat/sensors, wired to BhatBot via a new `home_control` tool (clean REST API; community MCP server exists).

- **What:** voice control of the physical space (M.I.L.E.S already does this).
- **Why now-ish:** natural fit for the Princeton dorm in the fall. Medium effort, qualitative jump.

D. Apple Watch glance app · EVERYDAY REACH

A native WatchKit app: last reply, tap-to-speak a short command, proactive notifications (brief, calendar, ambient triggers). Watch has mic + speaker → minimal voice path possible.

- **What:** rounds out surface coverage (wrist + phone + desktop + cloud).
- **Cost:** ~2–3 weeks of Swift. Payoff: the always-available feel.

E. Smart-glasses companion · MOONSHOT / ENDGAME

Frame-style glasses with mic, bone-conduction audio, and a small camera for visual context — BhatBot observes the physical environment (not just the screen) and answers in-ear.

- **What:** the honest "what JARVIS feels like in the films" answer — something worn, not an app.
- **Reality:** 6–12 months, requires hardware sourcing. Furthest out, most interesting.

Leverage summary

Rank	Item	Type	Effort	Status
1	Context-rot tool retrieval	SW	low	✅ W1
2	Cost/energy telemetry	SW	low	✅ W2
3	Key-risk auth tiers	SW	low	✅ W3
4	Knowledge-graph memory	SW	med	✅ W4
5	Local LoRA fine-tune loop	SW	high	✅ W5 (pipeline; run pending data)
6	Plugin sandboxing	SW	med	✅ W6
7	A2A handoff envelope	SW	low	✅ W7
8	Mac Mini M4 node	HW	buy	❌ deferred
9	ESP32 satellites	HW	low-\$	❌ deferred
10	Home Assistant	HW	med	❌ deferred
11	Apple Watch app	HW	weeks	❌ deferred
12	Smart glasses	HW	months	❌ moonshot

Generated alongside the W1–W7 implementation. The hardware tier is intentionally parked until the software foundation is exercised in daily use.